New Phone Purchase Decision Expert System

Detailed Technical Report

Name: Shoban Bhowmik

Student ID: 40325778

Course: COMP 474 – Intelligent Systems

April 24, 2026

Abstract

This report documents the complete architecture and knowledge base of a CLIPS-driven expert system for phone purchase decisions. The system reasons over current-device condition, replacement economics, and user preferences to produce one primary final decision, supporting evidence, confidence-rated outputs, warnings, a category suggestion, and optional concrete model examples from market facts. The implementation keeps all reasoning logic in CLIPS and uses Python only as a runner. This document covers every template, major function, rule family, interactive question, scenario profile, test expectation, and runtime behavior used by the project.

Contents

1 Project Goal and Problem Framing **3**

2 Implementation Architecture **3**

3 Knowledge Model: Complete Fact Templates **3**
3.1 Complete `phone-user` Slot Schema 4

4 Interactive Questionnaire: Questions and Conversions **4**

5 Inference Design and Rule Pipeline **5**
5.1 State Inference Rules 6
5.2 Finding Rules 6
5.3 Action Recommendation Rules 6
5.4 Certainty-Factor Rules 6
5.5 Fuzzy Evidence Rules 6

6 Final-Decision Layer **7**

7 Category and Model Suggestion Logic **7**

8 Scenario Profiles and Test Harness **7**
8.1 Scenario Intent 7
8.2 Test Assertions 7

9 Report Output Contract **8**

10 Market Data Pipeline **8**

11 Runtime Interface **8**

12 Rule and Template Counts **8**

13 Comprehensive Technical Inventory **8**
13.1 Complete Slot-Domain Specification for `phone-user` 9
13.2 Complete Interactive Prompt Sequence 9
13.3 Complete Scenario Fixture Definitions 10
13.4 Complete CLIPS Function Inventory 12
13.5 Complete Rule Catalog (All 55 Rules) 13

14 Limitations and Future Work **17**

15 Conclusion **17**

1 Project Goal and Problem Framing

The project addresses a practical decision problem: users often do not know whether they should keep a phone, wait, repair, replace the battery, or buy a replacement. The ambiguity is caused by mixed signals. A phone may still be software-supported but slow in daily use, or physically fine but expensive to repair relative to replacement. User context complicates this further because urgency, budget, workload, and feature expectations all influence the right action.

A symbolic expert system is suitable for this domain because decisions can be stated as explicit conditions, audited as rule traces, and validated through repeatable scenarios. The objective is not to "predict a product" but to provide explainable decision support with transparent reasoning steps.

2 Implementation Architecture

The codebase uses a CLIPS-first architecture. The Python program does not contain recommendation logic; it only initializes a CLIPS environment, loads knowledge files, and invokes CLIPS entry points.

The runtime path begins in `EXPERTSYSTEM/run.py`. This file imports `clipspy`, loads the CLIPS modules `clips/facts.clp`, `clips/market-facts.clp`, `clips/fuzzy.clp`, `clips/rules.clp`, and `tests/sample-scenarios.clp`, and then executes one mode: default run, named scenario run, scenario test suite, or interactive questionnaire mode.

The knowledge layer is split by responsibility. `facts.clp` defines all templates and a default `phone-user` profile. `fuzzy.clp` defines membership functions and a certainty-combination helper. `rules.clp` contains all inferencing behavior, output generation, and reporting format. `sample-scenarios.clp` provides both scenario fixtures and test harness logic.

3 Knowledge Model: Complete Fact Templates

The system currently defines 12 core templates. Their roles are summarized in Table 1.

Table 1: Template Inventory and Purpose

Template	Purpose and Structure
`phone-user`	Main case fact representing current device condition plus user preferences. Includes symbolic slots (for crisp matching) and numeric slots (for fuzzy evidence).
`recommendation`	Action-oriented supporting advice. Fields: `rule-id`, `message`.
`finding`	Observational evidence extracted from condition state. Fields: `rule-id`, `message`.
`warning`	Risk/caution outputs. Fields: `rule-id`, `message`.
`replacement-needed`	Intermediate inference fact indicating whether replacement pressure exists. Fields: `reason`, `strength`.

Template	Purpose and Structure
`no-upgrade-needed`	Intermediate blocking fact used to suppress buy-oriented paths when phone health is strong.
`final-decision`	Single primary decision target. Fields: `decision`, `rule-id`, `message`, `certainty`.
`advice-category-fired`	Duplicate-control marker used to avoid repeated evidence in the same category.
`recommendation-cf`	Confidence-weighted outputs from certainty-factor and fuzzy-evidence rules.
`phone-category-suggestion`	High-level market category recommendation aligned with final decision.
`phone-model-suggestion`	Concrete model candidates with rationale text (when relevant rule conditions are met).
`market-phone`	Preprocessed market fact record used by model suggestion rules. Includes source, segment, price, and spec/value fields.

3.1 Complete `phone-user` Slot Schema

The `phone-user` template encodes the full decision state through the following slots: `phone-age`, `battery-health`, `performance`, `storage-status`, `physical-condition`, `software-support`, `platform`, `repair-cost-level`, `budget-cad`, `budget-level`, `urgency`, `camera-need`, `gaming-need`, `work-study-dependence`, `latest-model-importance`, `used-phone-comfort`, `trade-in-available`, `battery-health-percent`, `battery-score`, `performance-score`, `budget-score`, `urgency-score`, and `repair-cost-percent`.

The design intentionally preserves both symbolic and numeric views of the same domain dimensions. Symbolic values are used for interpretable deterministic logic, while numeric values feed fuzzy functions and threshold-oriented certainty gates.

4 Interactive Questionnaire: Questions and Conversions

The interactive interview is fully implemented in CLIPS in `sample-scenarios.clp`. It asks the user explicit questions using `ask-choice` and `ask-int`, validates each response, and then derives symbolic categories from numeric fields before asserting one `phone-user` fact.

The exact interactive questions are:

1. Current phone age.
2. Current battery health percent.
3. Current performance score.

4. Current storage status.
5. Current physical condition.
6. Current software support status.
7. Current platform/ecosystem.
8. Estimated repair cost percent of replacement price.
9. Budget in CAD.
10. Urgency score for replacement.
11. Camera need level.
12. Gaming/performance need level.
13. Work/study dependence level.
14. Importance of latest model.
15. Comfort with used/refurbished phones (auto-set to `no` for high-budget level in current logic).
16. Trade-in availability.

After input, CLIPS applies conversion functions:

- `budget-cad-to-level`, `budget-cad-to-score`
- `battery-percent-to-health`
- `performance-score-to-level`
- `urgency-score-to-level`
- `repair-percent-to-level`

This approach keeps question phrasing practical for users while still giving rules the symbolic categories they require.

5 Inference Design and Rule Pipeline

The rule engine runs in a layered sequence. First, it infers state-control facts (replacement-needed vs. no-upgrade-needed). Second, it emits findings and supporting recommendations. Third, it emits certainty-factor and fuzzy-evidence recommendations. Fourth, it commits one final decision. Finally, it maps the final decision to category and model suggestions.

5.1 State Inference Rules

State inference rules establish whether the system should consider replacement actions at all. The `infer-no-upgrade-needed` rule requires a healthy condition profile (new/moderate age, good/moderate battery, fast/acceptable performance, good physical condition, supported software). Conversely, `replacement-needed` rules detect high or medium replacement pressure from combinations such as old + unsupported software, multiple major degradations, broken device with high repair level, or moderate-but-risky degradation patterns.

5.2 Finding Rules

The finding layer converts raw state into readable evidence facts. Current finding rules are: `F1` (very old phone), `F2` (poor battery), `F3` (slow performance), `F4` (full storage), `F5` (unsupported software/security). The battery finding uses `advice-category-fired` to avoid duplicate evidence in that category.

5.3 Action Recommendation Rules

Supporting recommendations are separate from final decisions. They include trade-in guidance (`R18`), refurbished path (`R17`), buy-new support (`R19`), keep-current support (`R20`), repair support (`R21`), avoid-unnecessary-flagship warning (`R22`), mid-range support (`R23`), flagship support (`R24`), and wait recommendation (`R25`).

Each of these rules is gated by the inferred state where appropriate. For example, buy-oriented rules require `replacement-needed` and are blocked by `no-upgrade-needed`.

5.4 Certainty-Factor Rules

The CF layer emits graded confidence outputs. The active CF rule IDs are: `BUY-NEW-CF`, `WAIT-CF`, `BATTERY-REPLACE-CF`, `REPAIR-CF`, `REFURB-CF`, `MID-RANGE-CF`, `FLAGSHIP-CF`, `KEEP-CURRENT-CF`.

These rules are intentionally strict. For example, `BATTERY-REPLACE-CF` requires poor battery health and suitable repair context; `REPAIR-CF` requires real device damage; `MID-RANGE-CF` and `FLAGSHIP-CF` require replacement-needed state and are blocked by no-upgrade-needed.

5.5 Fuzzy Evidence Rules

Fuzzy outputs are phrased as evidence signals, not final advice. The fuzzy rule IDs are: `FUZZY-BATTERY-POOR`, `FUZZY-PERFORMANCE-POOR`, `FUZZY-REPAIR-HIGH`, `FUZZY-BUDGET-URGENCY`, `FUZZY-WAIT`.

The membership functions are defined in `fuzzy.clp`. They are piecewise functions over bounded numeric domains for battery quality, performance quality, budget pressure, urgency pressure, and repair-cost pressure.

6 Final-Decision Layer

The system enforces one primary final decision through salience order and `(not (final-decision))`. Current final decision rule IDs are: `FINAL-KEEP`, `FINAL-REPLACE-BATTERY`, `FINAL-REPAIR`, `FINAL-BUY-FLAGSHIP`, `FINAL-BUY-MIDRANGE`, `FINAL-BUY-REFURB`, `FINAL-BUY-NEW`, `FINAL-WAIT`, `FINAL-FALLBACK`.

The current priority order is:

$$\texttt{KEEP} \rightarrow \texttt{REPLACE-BATTERY} \rightarrow \texttt{REPAIR} \rightarrow \texttt{BUY-FLAGSHIP} \rightarrow \texttt{BUY-MIDRANGE} \rightarrow \texttt{BUY-REFURB} \rightarrow \texttt{BUY-NEW} \rightarrow \texttt{WAIT} \rightarrow \texttt{FALLBACK}$$

This ordering is intended to prefer precise outcomes (for example flagship path) over broad outcomes (generic buy-new) when both are supported.

7 Category and Model Suggestion Logic

Category suggestions are always produced when a matching final decision exists, using `phone-category-suggestion`. These messages are concise and decision-oriented (for example keep-current review path, battery-service path, mid-range path, or premium/flagship upgrade path).

Model suggestions are generated by dedicated model rules (`M1`, `M4`, `M6`, `M8`, `M10`) against `market-phone` facts. The rules are constrained by final decision, segment, value/price/spec thresholds, and per-rule count caps through `count-model-suggestions`. If no models satisfy conditions, the report explicitly prints "None generated."

8 Scenario Profiles and Test Harness

The project includes five scenario profiles in `sample-scenarios.clp`. Each scenario asserts a full `phone-user` fact with symbolic and numeric slots.

8.1 Scenario Intent

Scenario-01 represents severe degradation with high replacement pressure. Scenario-02 represents a healthy phone with low urgency. Scenario-03 isolates battery-centric degradation. Scenario-04 models low budget with high urgency and meaningful wear. Scenario-05 models high-end usage demand with high budget and mixed condition.

8.2 Test Assertions

The harness validates four categories of behavior per scenario:

1. expected deterministic recommendation presence (when applicable),
2. expected CF rule presence,

3. expected final decision presence,
4. negative checks for forbidden CFs, forbidden categories, and forbidden final decisions.

This prevents false confidence where expected outputs appear but conflicting outputs also fire incorrectly.

9 Report Output Contract

The report printer in `rules.clp` always prints seven sections in fixed order: Final Decision, Key Findings, Supporting Recommendations, Confidence-Based Recommendations, Warnings, Suggested Phone Category, Suggested Phone Models.

Each section has explicit empty-state behavior ("None generated."). This makes output readable, stable for testing, and understandable to non-technical users.

10 Market Data Pipeline

The project uses two CSV sources as offline preprocessing input. The script `scripts/build_market_facts.py` selects/normalizes records and emits CLIPS facts into `clips/market-facts.clp`. At runtime, the inference engine reads this fact file directly.

The current pricing normalization assumption used by preprocessing is:

$$1 \text{ CAD} = 90 \text{ BDT}$$

11 Runtime Interface

From the `EXPERTSYSTEM/` directory, the supported commands are: `python3 run.py`, `python3 run.py -scenario scenario-01`, `python3 run.py -run-tests`, and `python3 run.py -interactive`.

The Python layer does not rank decisions or inject recommendation logic. Its responsibilities are argument parsing, CLIPS module loading, and invoking CLIPS entry points.

12 Rule and Template Counts

The current implementation contains 55 `defrule` rules in `rules.clp`. The template count in `facts.clp` is 12 core templates. This reflects a hybrid design with substantial symbolic coverage, explicit uncertainty modeling, and structured output control.

13 Comprehensive Technical Inventory

This section is an exhaustive reference appendix intended to let the project be understood without opening source files. It enumerates all slot domains, all interactive prompts, all scenario fixtures, all

CLIPS functions, and every active rule with trigger and output summaries.

13.1 Complete Slot-Domain Specification for `phone-user`

Table 2 gives every slot in `phone-user` exactly as defined in `facts.clp`.

Table 2: Full `phone-user` Slot Domains

Slot	Domain / Constraint
`phone-age`	`new, moderate, old, very-old`
`battery-health`	`good, moderate, poor, very-poor`
`performance`	`fast, acceptable, slow, very-slow`
`storage-status`	`enough, almost-full, full`
`physical-condition`	`good, minor-damage, major-damage, broken`
`software-support`	`supported, limited, unsupported`
`platform`	`ios, android, other`
`repair-cost-level`	`low, medium, high, very-high`
`budget-cad`	integer in `[0, 20000]`
`budget-level`	`low, medium, high`
`urgency`	`low, medium, high`
`camera-need`	`low, medium, high`
`gaming-need`	`low, medium, high`
`work-study-dependence`	`low, medium, high`
`latest-model-importance`	`low, medium, high`
`used-phone-comfort`	`yes, no, maybe`
`trade-in-available`	`yes, no`
`battery-health-percent`	integer in `[0, 100]`
`battery-score`	integer in `[0, 100]`
`performance-score`	integer in `[0, 100]`
`budget-score`	integer in `[0, 100]`
`urgency-score`	integer in `[0, 100]`
`repair-cost-percent`	integer in `[0, 100]`

13.2 Complete Interactive Prompt Sequence

The interactive questionnaire in `sample-scenarios.clp` asks the following prompts in this exact order:

Prompt	Input Type
Current phone age	choice
Current battery health percent	integer
Current performance score	integer
Current storage status	choice
Current physical condition	choice
Current software support status	choice
Current platform/ecosystem	choice
Estimated repair cost percent of replacement price	integer
Your budget in CAD	integer
How urgent is replacement for you (score)	integer
Your camera need level	choice
Your gaming/performance need level	choice
How much you depend on phone for work/study	choice
Importance of getting latest model	choice
Comfort with used/refurbished phones	choice (or auto-set to `no` in high-budget branch)
Trade-in available for current phone	choice

After data collection, conversion functions derive symbolic categories from numeric inputs:

- `budget-cad-to-level`
- `budget-cad-to-score`
- `battery-percent-to-health`
- `performance-score-to-level`
- `urgency-score-to-level`
- `repair-percent-to-level`

13.3 Complete Scenario Fixture Definitions

The test harness defines five complete `phone-user` fixture assertions. For traceability, each scenario's full fact values are listed in Table 4.

Table 4: Scenario Fixture Definitions from `assert-scenario`

Scenario	Complete `phone-user` Values
scenario-01	phone-age=very-old, battery-health=very-poor, performance=very-slow, storage-status=full, physical-condition=minor-damage, software-support=unsupported, platform=android, repair-cost-level=high, budget-cad=1000, budget-level=medium, urgency=high, camera-need=medium, gaming-need=low, work-study-dependence=high, latest-model-importance=low, used-phone-comfort=maybe, trade-in-available=yes, battery-health-percent=15, battery-score=15, performance-score=20, budget-score=60, urgency-score=90, repair-cost-percent=75.
scenario-02	phone-age=moderate, battery-health=good, performance=fast, storage-status=enough, physical-condition=good, software-support=supported, platform=android, repair-cost-level=low, budget-cad=800, budget-level=medium, urgency=low, camera-need=medium, gaming-need=low, work-study-dependence=medium, latest-model-importance=low, used-phone-comfort=no, trade-in-available=no, battery-health-percent=85, battery-score=85, performance-score=88, budget-score=55, urgency-score=20, repair-cost-percent=10.
scenario-03	phone-age=moderate, battery-health=very-poor, performance=acceptable, storage-status=enough, physical-condition=good, software-support=supported, platform=android, repair-cost-level=medium, budget-cad=700, budget-level=medium, urgency=medium, camera-need=low, gaming-need=low, work-study-dependence=high, latest-model-importance=low, used-phone-comfort=maybe, trade-in-available=no, battery-health-percent=18, battery-score=18, performance-score=70, budget-score=55, urgency-score=50, repair-cost-percent=25.

Scenario	Complete phone-user Values
scenario-04	phone-age=old, battery-health=poor, performance=slow, storage-status=almost-full, physical-condition=minor-damage, software-support=limited, platform=android, repair-cost-level=high, budget-cad=400, budget-level=low, urgency=high, camera-need=medium, gaming-need=low, work-study-dependence=high, latest-model-importance=low, used-phone-comfort=yes, trade-in-available=yes, battery-health-percent=30, battery-score=30, performance-score=35, budget-score=25, urgency-score=85, repair-cost-percent=65.
scenario-05	phone-age=old, battery-health=moderate, performance=slow, storage-status=almost-full, physical-condition=good, software-support=limited, platform=ios, repair-cost-level=medium, budget-cad=1800, budget-level=high, urgency=medium, camera-need=high, gaming-need=high, work-study-dependence=high, latest-model-importance=high, used-phone-comfort=no, trade-in-available=yes, battery-health-percent=55, battery-score=55, performance-score=45, budget-score=90, urgency-score=65, repair-cost-percent=35.

13.4 Complete CLIPS Function Inventory

Table 5 lists every defined `deffunction` in the project logic files.

Table 5: Complete Function Inventory

Function	File	Role
fuzzy-battery-poor, fuzzy-battery-moderate, fuzzy-battery-good	fuzzy.clp	battery memberships
fuzzy-performance-poor, fuzzy-performance-good	fuzzy.clp	performance memberships
fuzzy-budget-low, fuzzy-budget-medium, fuzzy-budget-high	fuzzy.clp	budget memberships
fuzzy-urgency-low, fuzzy-urgency-high	fuzzy.clp	urgency memberships
fuzzy-repair-cost-high	fuzzy.clp	repair-cost membership

Function	File	Role
`combine-cf`	fuzzy.clp	CF combination helper
`count-model-suggestions`	rules.clp	model cap helper by rule-id
`print-results`	rules.clp	ordered report printer
`clear-phone-users`, `ask-choice`, `ask-int`	sample-scenarios.clp	interaction helpers
`budget-cad-to-level`, `budget-cad-to-score`	sample-scenarios.clp	budget conversion
`battery-percent-to-health`, `performance-score-to-level`	sample-scenarios.clp	condition conversion
`urgency-score-to-level`, `repair-percent-to-level`	sample-scenarios.clp	urgency/repair conversion
`run-interactive`, `assert-scenario`, `run-scenario`, `run-scenario-silent`	sample-scenarios.clp	execution entry points
`expected-primary-rule`, `expected-cf-rule`, `expected-final-rule`	sample-scenarios.clp	test expectation mapping
`recommendation-exists`, `recommendation-cf-exists`, `final-decision-exists`, `cf-not-exists`, `phone-category-exists`	sample-scenarios.clp	fact existence checks
`test-scenario-negative-cf`, `test-scenario-negative-category`, `test-scenario-negative-final`, `test-scenario`, `run-all-scenario-tests`	sample-scenarios.clp	regression testing logic

13.5 Complete Rule Catalog (All 55 Rules)

Table 6 lists every `defrule` in `rules.clp` with a concise trigger-and-output summary.

Table 6: Complete Rule Catalog (rules.clp)

Rule Name	Trigger Summary	Asserted Output
`infer-no-upgrade-needed`	healthy modern profile	`no-upgrade-needed`
`infer-replacement-needed-multiple-issues`	old + poor battery + slow performance	`replacement-needed(high)`
`infer-replacement-needed-old-unsupported`	old/very-old + unsupported software	`replacement-needed(high)`
`infer-replacement-needed-broken-expensive`	broken/major damage + high repair level	`replacement-needed(high)`
`infer-replacement-needed-medium-degradation`	moderate-old degradation mix	`replacement-needed(medium)`
`infer-replacement-needed-medium-performance`	weak performance + limited support	`replacement-needed(medium)`
`finding-very-old-phone`	phone-age very-old	`finding(F1)`
`finding-poor-battery`	poor battery and no battery marker	`finding(F2)` + marker
`finding-slow-performance`	slow/very-slow performance	`finding(F3)`
`finding-full-storage`	storage full	`finding(F4)`
`finding-unsupported-software`	unsupported software	`finding(F5)`
`rule-18-trade-in`	trade-in available yes	`recommendation(R18)`
`rule-17-refurbished-option`	replacement-needed + low/medium budget + used comfort	`recommendation(R17)`
`rule-19-buy-new-phone`	replacement-needed high + medium/high budget	`recommendation(R19)`
`rule-20-keep-current-phone`	no-upgrade-needed + urgency low/medium	`recommendation(R20)`
`rule-21-repair-instead`	damage + viable repair + usable performance/support	`recommendation(R21)`
`rule-22-avoid-flagship`	low/medium budget + high latest-model desire + moderate needs	`warning(R22)`
`rule-23-buy-mid-range`	replacement-needed + medium budget + balanced needs	`recommendation(R23)`
`rule-24-buy-flagship`	replacement-needed + high budget + high needs	`recommendation(R24)`
`rule-25-wait-before-buying`	no replacement-needed + low urgency + usable phone	`recommendation(R25)`
`warning-unsupported-software`	unsupported software	`warning(R8)`

Rule Name	Trigger Summary	Asserted Output
`cf-buy-new-phone`	replacement-needed + multi-factor score build	`recommendation-cf(BUY-NEW-CF)`
`cf-wait-before-buying`	no high replacement-needed + healthy current profile	`recommendation-cf(WAIT-CF)`
`cf-replace-battery`	poor battery + low repair-percent + usable rest	`recommendation-cf(BATTERY-REPLACE-CF)`
`cf-repair-instead-of-buy`	damage + low/medium repair + usable rest	`recommendation-cf(REPAIR-CF)`
`cf-buy-refurbished`	replacement-needed + low/medium budget + used comfort	`recommendation-cf(REFURB-CF)`
`cf-buy-mid-range`	replacement-needed + medium budget + balanced needs	`recommendation-cf(MID-RANGE-CF)`
`cf-buy-flagship`	replacement-needed + high budget + high needs	`recommendation-cf(FLAGSHIP-CF)`
`cf-keep-current`	no-upgrade-needed profile	`recommendation-cf(KEEP-CURRENT-CF)`
`rule-fuzzy-poor-battery`	fuzzy poor-battery membership > 0.30	`recommendation-cf(FUZZY-BATTERY-POOR)`
`rule-fuzzy-poor-performance`	fuzzy poor-performance membership > 0.30	`recommendation-cf(FUZZY-PERFORMANCE-POOR)`
`rule-fuzzy-high-repair-cost`	fuzzy high-repair membership > 0.30	`recommendation-cf(FUZZY-REPAIR-HIGH)`
`rule-fuzzy-high-budget-high-urgency`	fuzzy min(high-budget, high-urgency) > 0.30	`recommendation-cf(FUZZY-BUDGET-URGENCY)`
`rule-fuzzy-low-urgency`	no high replacement-needed + fuzzy low urgency > 0.30	`recommendation-cf(FUZZY-WAIT)`
`final-decision-buy-new`	BUY-NEW-CF >= 0.70	`final-decision(FINAL-BUY-NEW)`
`final-decision-keep-current`	KEEP-CURRENT-CF >= 0.70 and not dominated by BUY-NEW-CF	`final-decision(FINAL-KEEP)`
`final-decision-replace-battery`	BATTERY-REPLACE-CF >= 0.70	`final-decision(FINAL-REPLACE-BATTERY)`
`final-decision-repair`	REPAIR-CF >= 0.70	`final-decision(FINAL-REPAIR)`
`final-decision-buy-flagship`	FLAGSHIP-CF >= 0.70 and not dominated by BUY-NEW-CF	`final-decision(FINAL-BUY-FLAGSHIP)`

Rule Name	Trigger Summary	Asserted Output
final-decision-buy-midrange	MID-RANGE-CF >= 0.65 and not dominated by BUY-NEW-CF	final-decision(FINAL-BUY-MIDRANGE)
final-decision-buy-refurb	REFURB-CF >= 0.60 and not dominated by BUY-NEW-CF	final-decision(FINAL-BUY-REFURB)
category-keep-current	final decision keep-current/wait	phone-category-suggestion(C-KEEP)
category-repair	final decision repair	phone-category-suggestion(C-REPAIR)
category-replace-battery	final decision replace-battery	phone-category-suggestion(C-BATTERY)
category-mid-range	final decision buy-mid-range	phone-category-suggestion(C-MID)
category-refurb	final decision buy-refurbished	phone-category-suggestion(C-REFURB)
category-flagship	final decision buy-flagship	phone-category-suggestion(C-FLAG)
category-buy-new	final decision buy-new + budget branch	phone-category-suggestion(C-*UPGRADE)
final-decision-wait	WAIT-CF >= 0.70	final-decision(FINAL-WAIT)
final-decision-fallback	no final decision + supporting keep/wait recommendation	final-decision(FINAL-FALLBACK)
model-flagship-camera-from-data	final decision buy-flagship/buy-new + flagship camera fit + count cap	phone-model-suggestion(M1)
model-midrange-value-from-data	final decision buy-mid-range/buy-new + value mid-range fit + count cap	phone-model-suggestion(M4)
model-budget-from-data	upgrade path + low budget + budget segment fit + count cap	phone-model-suggestion(M6)
model-refurb-estimate-from-market	final decision buy-refurbished + used comfort + premium/flagship fit + cap	phone-model-suggestion(M8)

Rule Name	Trigger Summary	Asserted Output
`model-wait-watchlist-from-data`	final decision wait + high-spec flagship fit + count cap	`phone-model-suggestion(M10)`

14 Limitations and Future Work

The system remains deterministic-plus-heuristic rather than learned from historical outcomes. Market data is snapshot-based and not live. Some real-world constraints (financing behavior, carrier lock-in, long-term ecosystem switching cost) are not modeled explicitly. Future work can add these constraints and refine top-k model diversification while preserving explainability.

15 Conclusion

The project now represents a full expert-system workflow: structured input acquisition, layered inferencing, conflict control, single final decision selection, auditable supporting evidence, category/model suggestion output, and scenario regression testing with negative checks. The design remains faithful to COMP 474 principles by keeping reasoning in CLIPS and maintaining transparent rule-based logic throughout.

References

1. Showmik121, *Smartphones Dataset 2026 (1000 devices)*, Kaggle.
   https://www.kaggle.com/datasets/showmik121/smartphones-dataset-2026-1000-devices

2. Userdominator, *Mobile Phone Price Dataset*, Kaggle.
   https://www.kaggle.com/datasets/userdominator/mobile-phone-price-dataset